Relojes

Qué pasó primero, sin un reloj perfecto

LA PREGUNTA DE HOY

Si dos computadoras no comparten un reloj perfecto, ¿cómo sabemos qué pasó primero?



Relojes físicos

Causalidad

[2,4,1]

Relojes lógicos

Entrega

00 / APERTURA

Cada máquina tiene su propio ahora.

Log imposible

¿El dinero llegó antes de salir?

Juntamos los logs de dos servidores y los ordenamos por hora.
El efecto aparece cinco segundos antes que su causa.

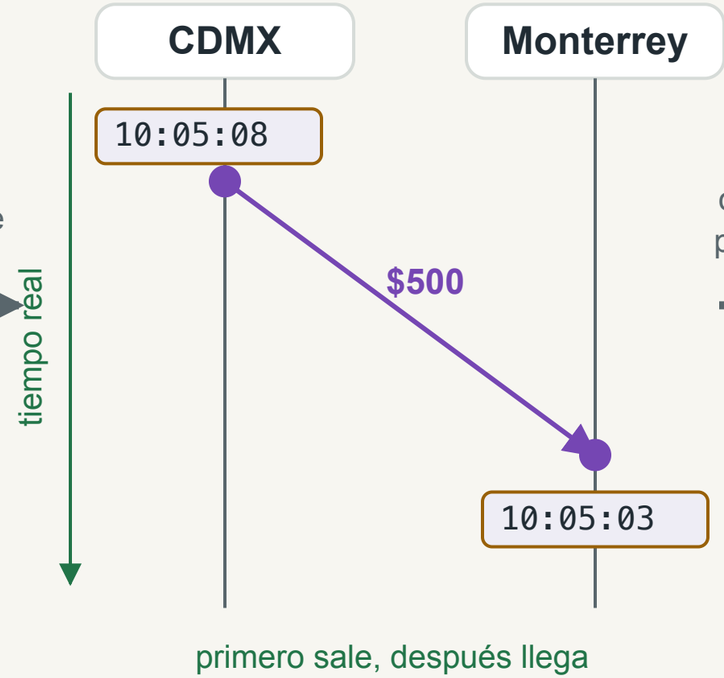
Mexi Banco · CDMX

```
10:05:08
transferencia enviada
```

Mexi Banco · Monterrey

```
10:05:03
transferencia recibida
```

lo que pasó



ordenar por hora

LOG CENTRAL · ORDENADO POR HORA

```
10:05:03 MTY
transferencia recibida
```

```
10:05:08 CDMX
transferencia enviada
```

efecto antes que causa

✗ Recibida 5 s antes de enviarse

✗ Nadie marca error:
el log solo miente

⚠ **El mensaje baja en el tiempo, pero el reloj de Monterrey dice que subió.**

Ganador equivocado

¿Quién compró primero la última butaca?

Si el sistema decide el orden comparando las marcas de relojes distintos, 120 ms de desfase bastan para darle la butaca a quien llegó después.



1. **Millones compran a la vez**
2. **Cada región, su reloj**

   Londres va 120 ms adelantado
3. **Regla: gana la marca menor**
4. **Gana quien no era**

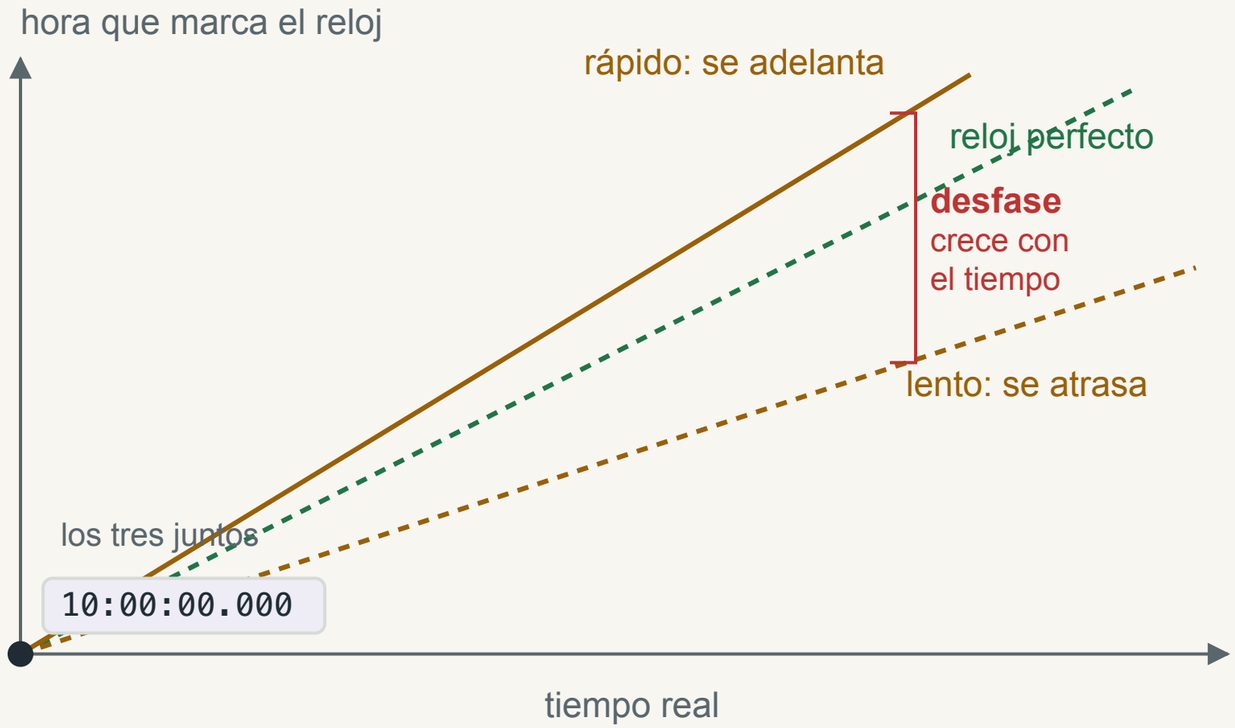
   y nadie se entera

Comparar horas de máquinas distintas es comparar relojes, no eventos.

Deriva

¿Por qué dos relojes que empiezan juntos terminan separados?

Cada máquina cuenta el tiempo con un cristal de cuarzo, y no hay dos iguales.
Cada reloj avanza a su propio ritmo: unos se adelantan, otros se atrasan.



1 CUARZO

Un cristal vibra a un ritmo casi fijo; el sistema cuenta vibraciones.

2 ¿POR QUÉ DERIVA?

temperatura · humedad · edad

3 ¿CUÁNTO?

1 ppm ≈ 86 ms al día
200 ppm: 6 ms cada 30 s, o 17 s al día

Todo reloj local miente un poco, y si nadie lo corrige, miente más cada día.

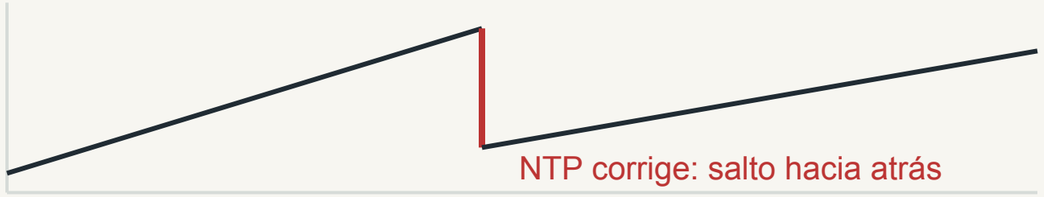
Pared y monotónico

¿Qué reloj leo para qué?

El reloj de pared da la fecha y hora (UTC) y puede saltar hacia atrás.
El monotónico solo avanza: sirve para medir duraciones, no para comparar máquinas.

Reloj de pared

`System.currentTimeMillis()` `CLOCK_REALTIME`



✓ Fecha y hora: logs, vencimientos, agenda

✗ Puede ir hacia atrás

Reloj monotónico

`System.nanoTime()` `CLOCK_MONOTONIC`



✓ Duraciones y timeouts

✗ Su valor solo no significa nada:
no se compara entre máquinas

Medir una duración con el reloj de pared: NTP salta 2 s atrás

`inicio = 10:05:08.000` → `fin = 10:05:06.500` `fin − inicio = −1.5 s`

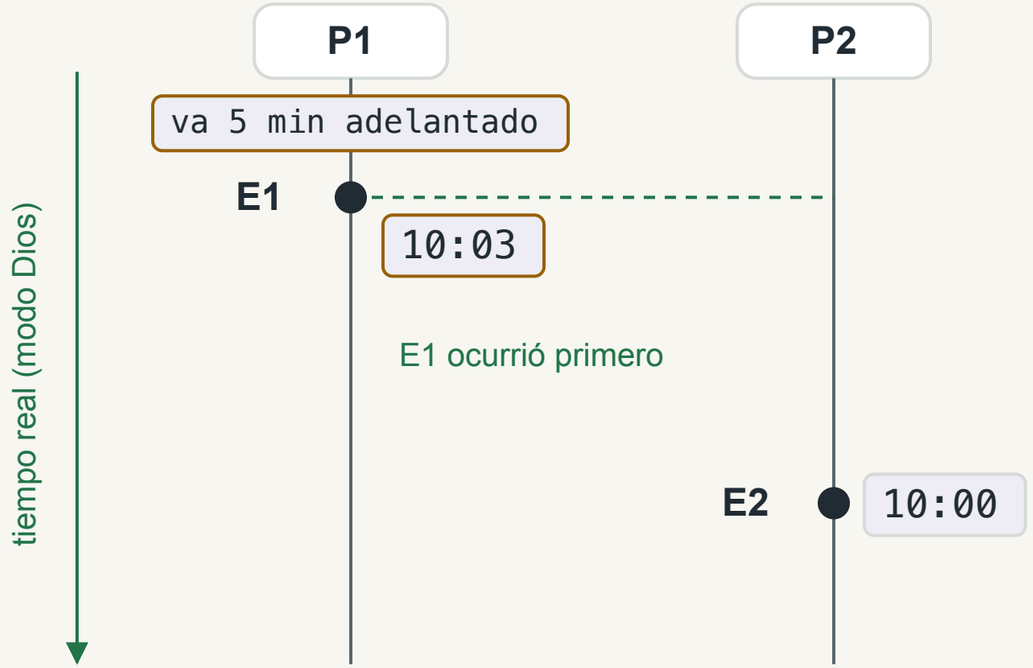
✓ con nanoTime(): 0.5 s

Para medir cuánto tardó, el monotónico; para decir cuándo pasó, el de pared.

Desfase

¿Puedo ordenar eventos de dos máquinas por su hora?

Desfase (clock skew, ε): la diferencia entre lo que marcan dos relojes en el mismo instante.
Si lo conociera exacto, lo restaría; el problema es que no se puede medir exacto.



un tercero ordena por hora

ORDEN POR HORA

1. 10:00 E2
2. 10:03 E1

✗ Orden invertido

Si conozco ε, lo resto

`10:03 − 5 min = 9:58`

✓ 9:58 < 10:00: E1 primero

Pero ε no se mide exacto.

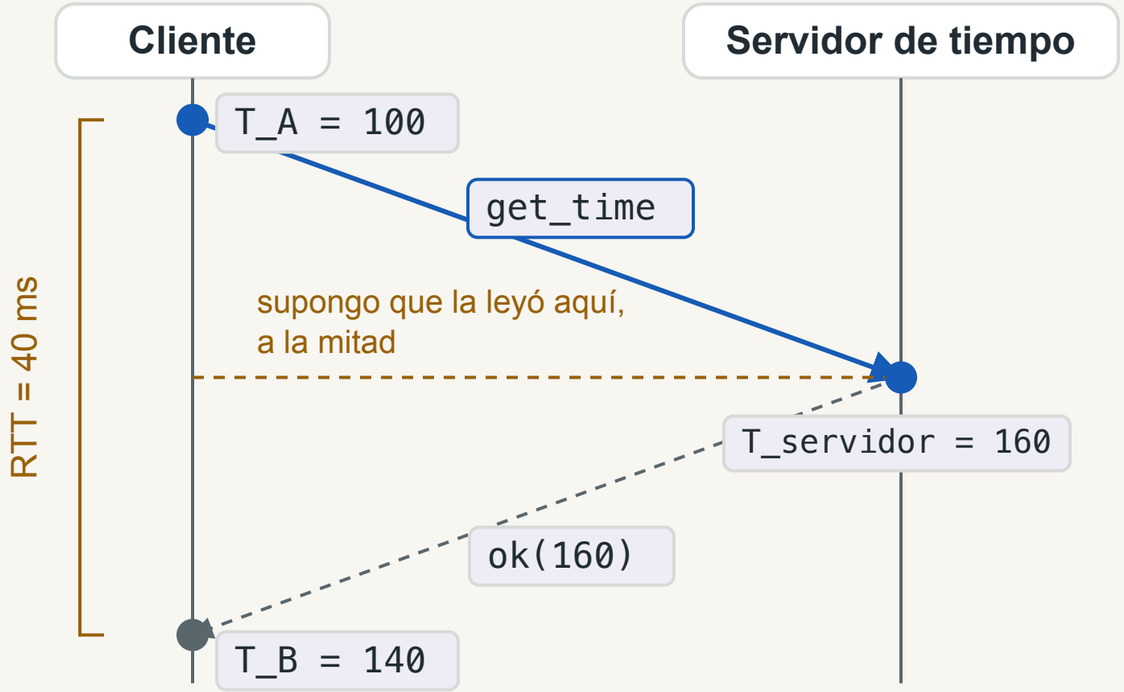
Para conocerlo hay que preguntar, y preguntar tarda.

Ordenar por hora solo funciona si sabes cuánto miente cada reloj.

Cristian

¿Cómo sé qué hora es en la otra máquina?

Pregunto la hora, mido la ida y vuelta (RTT) y supongo que me la leyeron a la mitad: hora estimada = T_servidor + RTT / 2. El error queda dentro de ± RTT / 2.

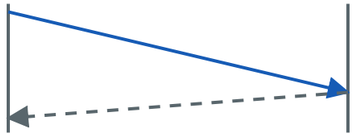


1 HORA ESTIMADA

```
hora = T_servidor + RTT / 2
= 160 + 40 / 2 = 180
```

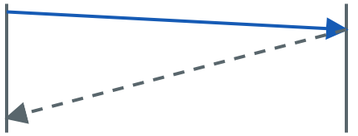
Mi reloj marcaba 140: voy 40 ms atrasado.

2 Ida 35 ms, vuelta 5 ms



desfase real: 25 ms

3 Ida 5 ms, vuelta 35 ms



desfase real: 55 ms

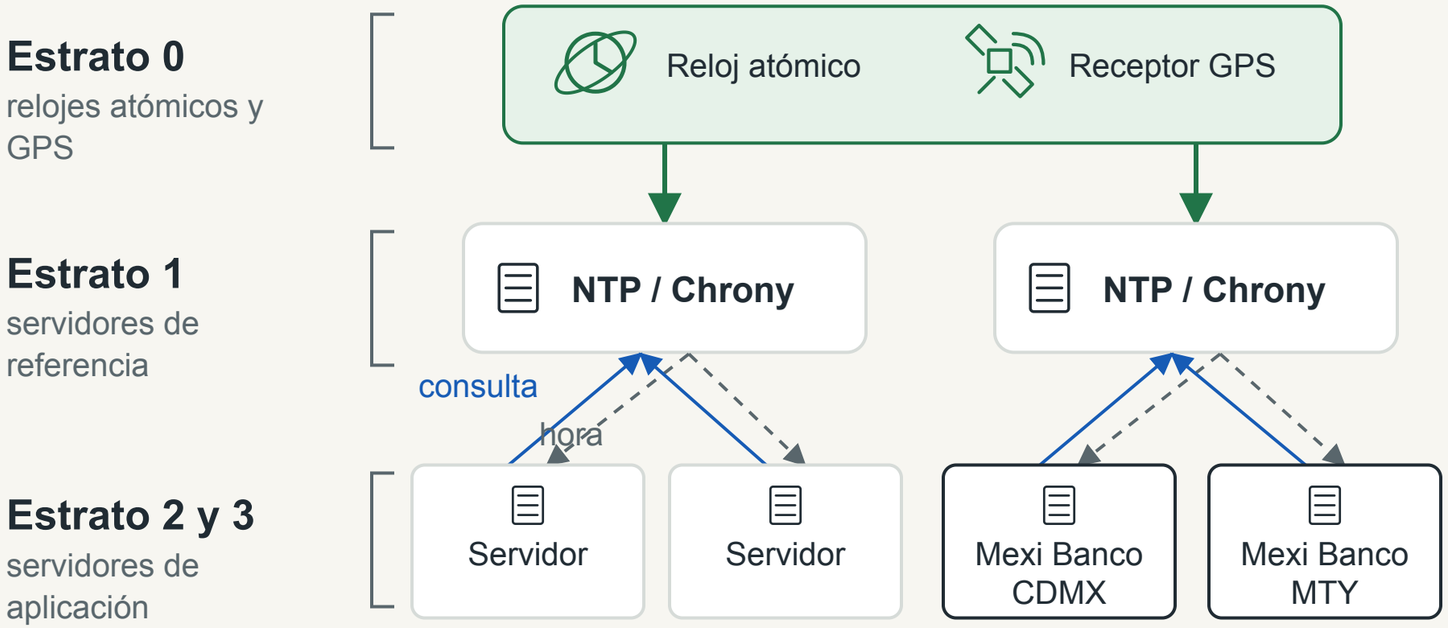
Mismo RTT, Cristian dice 40 en los dos: error ± 20 ms

Puedo estimar la hora del otro, pero me equivoco hasta la mitad del viaje.

NTP

¿Y si miles de máquinas necesitan la hora?

Network Time Protocol: una jerarquía de servidores (estratos) baja la hora UTC desde relojes atómicos y GPS, y cada cliente corrige su reloj poco a poco.



1 Sondeo continuo

Cada cliente pregunta a los de arriba, una y otra vez.

2 Desfase y retardo

Los calcula en cada sondeo (siguiente lámina).

3 Slew, no salto

Acelera o frena el reloj poco a poco (hasta 500 ppm).



4 Step si el desfase es grande

En ntpd, más de 128 ms: salta, y el reloj de pared puede ir atrás.

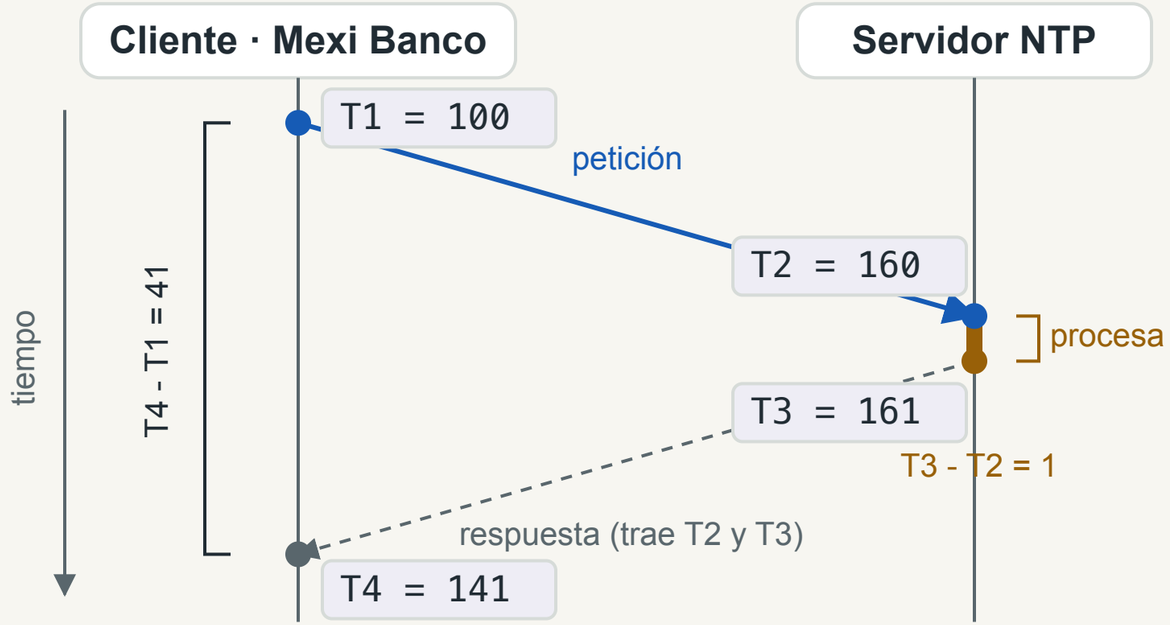


NTP no pone la hora una vez: la mide y la corrige todo el tiempo.

Cuatro marcas

¿Cómo calcula NTP el desfase si el mensaje tarda?

En cada sondeo se anotan cuatro marcas: T1 y T4 con el reloj del cliente, T2 y T3 con el del servidor.
Con ellas salen el desfase θ y el retardo δ, sin contar lo que tardó el servidor.



valores en ms; T1 y T4 en el reloj del cliente,
T2 y T3 en el del servidor

1 Desfase θ

$$\theta = ((T2 - T1) + (T3 - T4)) / 2$$
$$= ((160 - 100) + (161 - 141)) / 2$$
$$= (60 + 20) / 2 = 40 \text{ ms}$$

El cliente va 40 ms atrasado: hay que adelantarlo.

2 Retardo δ

$$\delta = (T4 - T1) - (T3 - T2) = 41 - 1 = 40 \text{ ms}$$

Tiempo en la red, ida y vuelta, sin contar al servidor.

⚠ **Supone que la ida y la vuelta tardan lo mismo.**
Si no, el error llega hasta ± δ / 2 = ± 20 ms.

Cuatro marcas cancelan el viaje si es simétrico; si no, queda un error de hasta δ / 2.

Incertidumbre

¿Por qué ni con NTP los relojes coinciden?

Tres cosas impiden la sincronía perfecta: el cuarzo sigue derivando,
la red no tarda lo mismo de ida que de vuelta, y a UTC a veces le agregan un segundo.



Ventana de incertidumbre

± ε

1 DERIVA

Entre un sondeo y otro, el cuarzo
se vuelve a separar.

2 LATENCIA VARIABLE

La ida y la vuelta no tardan lo mismo, y eso no
se mide desde los extremos: error hasta ± δ / 2.

3 SEGUNDO INTERCALAR

A veces UTC agrega un segundo y 23:59:59 se repite.
Google y AWS lo reparten en 24 h (smearing).

Falla en silencio:
no hay error de reloj,
hay resultados equivocados.

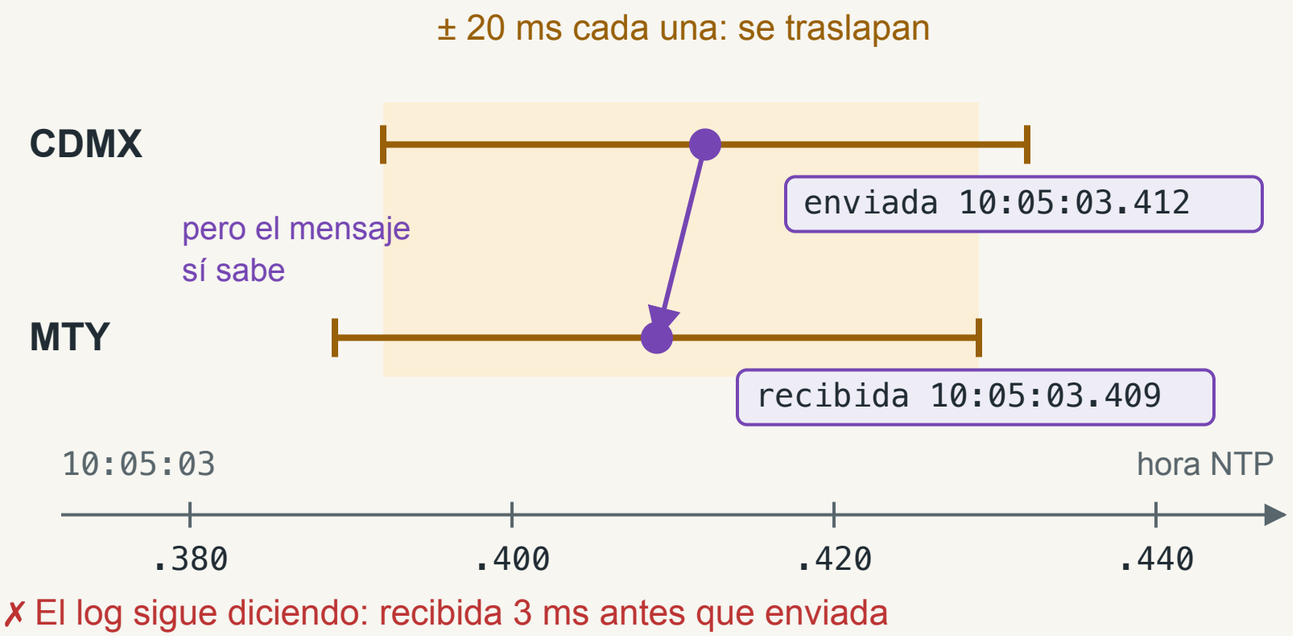
NTP achica el desfase, pero no lo vuelve cero ni nos dice de qué lado está.

Hora contra orden

Si todos usan NTP, ¿ya resolvimos el orden?

Con NTP el log de Mexi Banco pasa de 5 s al revés a 3 ms al revés: mejor, pero sigue al revés.
Dos eventos más cercanos que el error no tienen orden seguro por hora.

EL MISMO LOG, AHORA CON NTP



✗ El log sigue diciendo: recibida 3 ms antes que enviada

¿Qué hora es?

Relojes físicos: siempre ± ε.

cambiamos la pregunta

¿Qué pasó primero?

Causalidad: el envío va antes que su recepción, siempre.

Afinar los relojes no ordena los eventos: para eso necesitamos causalidad.

Tiempo lógico

¿Y si dejamos de preguntar qué hora es y preguntamos qué pasó primero?

Tiempo lógico: ordenar los eventos por lo que pudo influir en qué, sin leer ningún reloj.
Importan números que reflejen el orden, no la hora real.

LA PREGUNTA DEL BLOQUE ANTERIOR

¿Qué hora es?

Con NTP: casi.

✗ ε se estima; nunca se conoce exacto

✗ La hora puede contradecir la causa

cambio de pregunta

LA PREGUNTA NUEVA



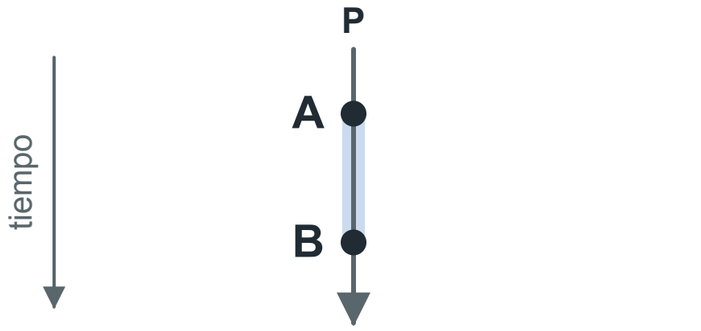
Ya no preguntamos qué hora es, sino qué pudo influir en qué.

Happens-before

¿Cuándo podemos asegurar que A pasó antes que B, sin mirar el reloj?

A → B («A pasa antes que B»): A pudo influir en B. Un evento es un paso local, un envío o una recepción. Lamport la define con tres reglas, y ninguna lee un reloj.

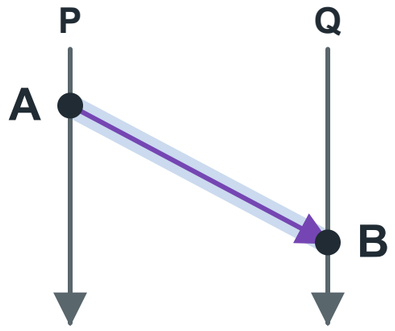
1 Mismo proceso



Si A y B están en el mismo proceso y A ocurre primero, A → B.

A → B

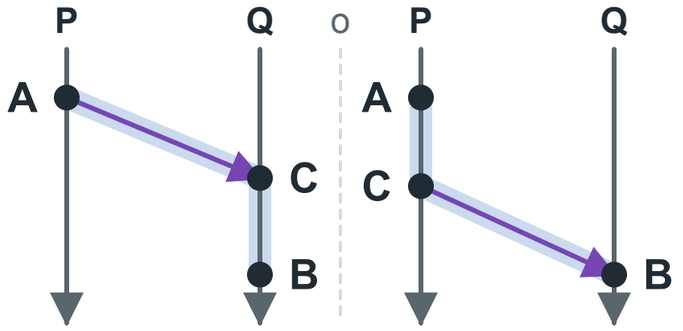
2 Mensaje



Si A es el envío de un mensaje y B su recepción, A → B.

A → B

3 Transitividad



Si A → C y C → B, entonces A → B.

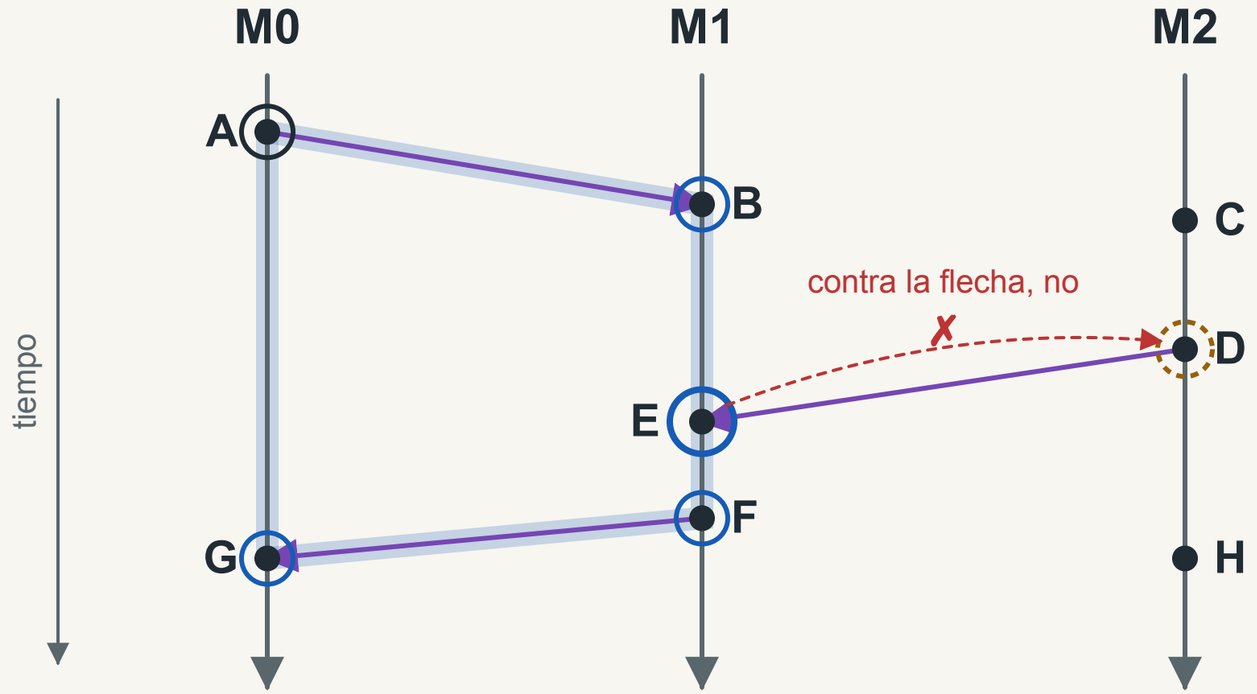
A → B

Tres reglas y ningún reloj: el orden sale de los procesos y los mensajes.

Seguir las flechas

¿A pudo influir en E? ¿Y en D?

Si desde A llegas a B bajando por los procesos y avanzando por los mensajes, A → B.
Si no hay camino en ningún sentido, son concurrentes: A ‖ B.



1 ¿A y E? A → E

A → B → E

por el mensaje, y luego en M1

2 ¿A y D? A ‖ D

Desde A: B, E, F, G. Nunca D.

Desde D: E, F, G, H. Nunca A.

Sin camino en ningún sentido.

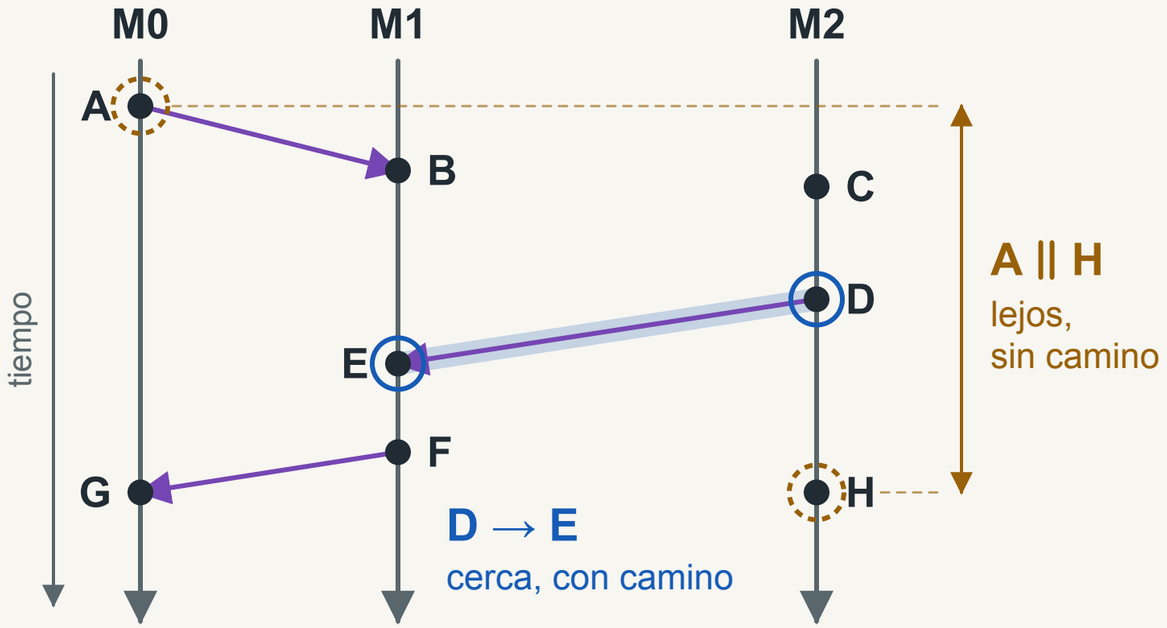
Hay camino: A → B. No hay camino en ningún sentido: A ‖ B.

Concurrente

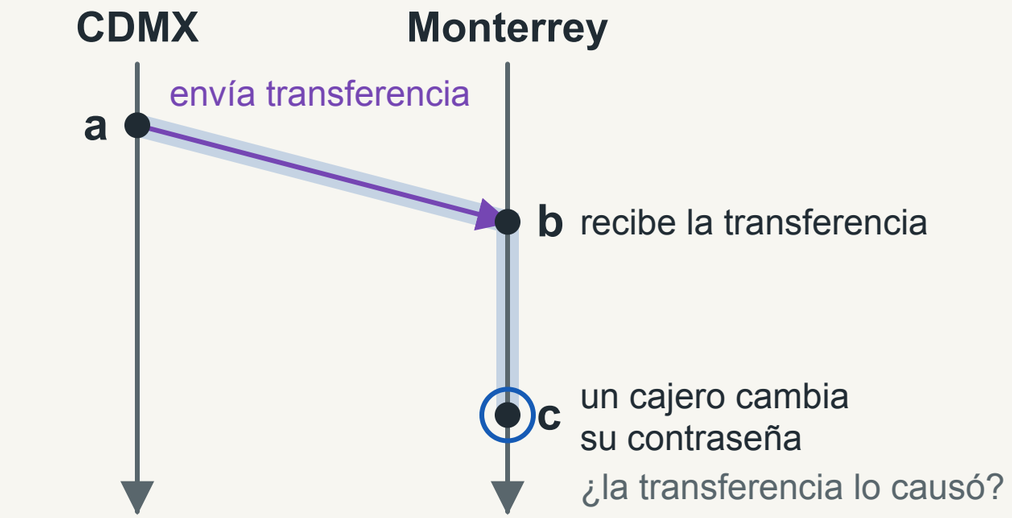
¿Concurrente quiere decir «al mismo tiempo»?

Concurrentes: no hay camino entre ellos, así que ninguno pudo influir en el otro.
Y A → B dice que A pudo influir en B, no que lo hizo: causalidad potencial.

CONCURRENTE NO ES «AL MISMO TIEMPO»



A → B QUIERE DECIR «PUDO INFLUIR»



✓ a → c: a pudo influir en c
✗ No dice que a causó c

|| Concurrente quiere decir sin camino, no al mismo tiempo.

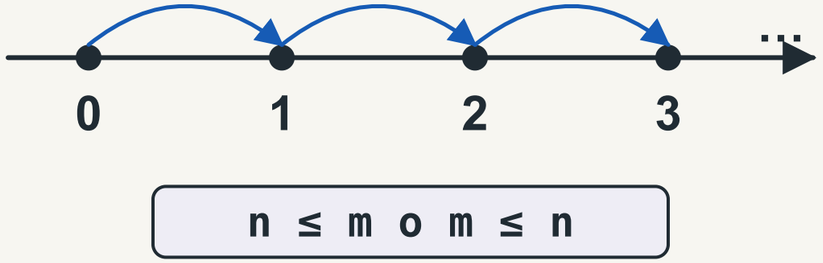
Orden parcial

¿Se pueden formar todos los eventos en una sola fila?

Orden total: todos se comparan con todos. Orden parcial: solo algunos pares son comparables. Happens-before es un orden parcial: los pares concurrentes no se comparan.

ORDEN TOTAL

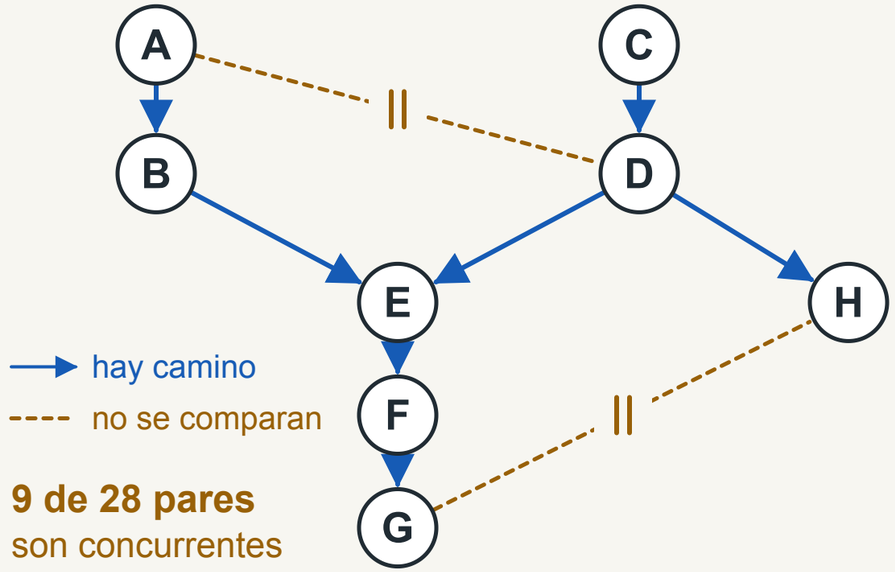
los números naturales



✓ Cualquier par se compara

Todo orden total también es parcial.

ORDEN PARCIAL · LOS 8 EVENTOS DE A A H



¿un número por evento?

A = ?
B = ?
C = ?
D = ?
E = ?
F = ?
G = ?
H = ?

? ¿Cómo le pongo un número a cada evento que respete las flechas?

Reloj de Lamport

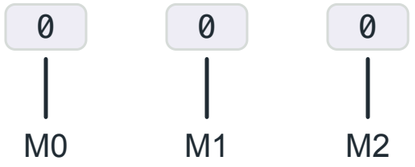
¿Y si en vez de contar segundos contamos eventos?

Un contador por proceso que avanza con cada evento y viaja en cada mensaje.
No dice la hora ni cuánto tiempo pasó: solo sirve para ordenar.

1 Empieza en 0

`C = 0`

Cada proceso guarda su propio contador.



2 Cada evento suma 1

`C = C + 1`

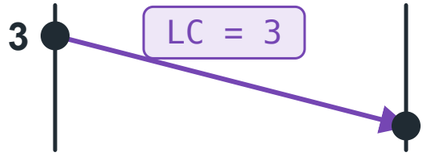
Internos, envíos y recepciones: todos cuentan.



3 Al enviar, viaja C

`enviar(m, C)`

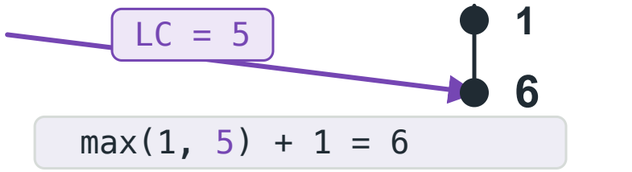
El mensaje lleva el contador ya incrementado.



4 Al recibir, máximo + 1

`C = max(local, recibido) + 1`

Quien recibe nunca queda por debajo de lo que llegó.



Hoy la recepción cuenta como evento: por eso el + 1.

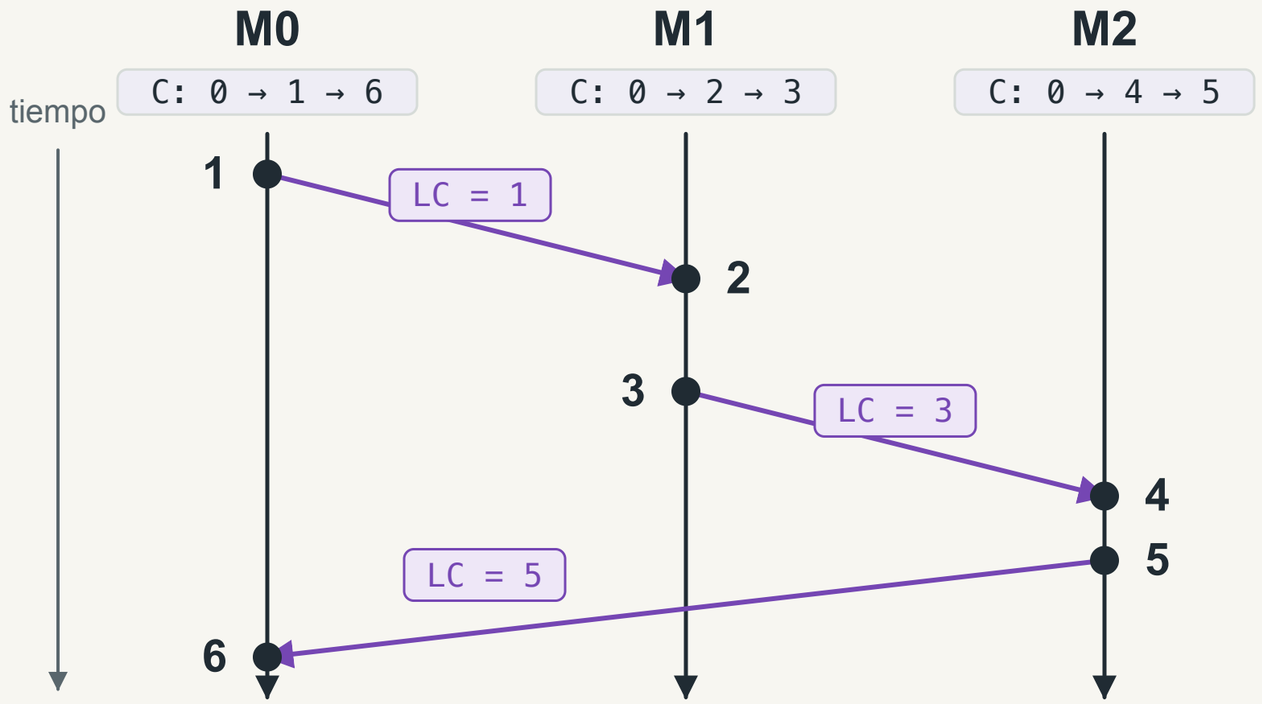
Algunos textos no la cuentan y escriben la regla sin el + 1.

Cuenta eventos, manda el número y, al recibir, toma el máximo más uno.

La cadena

¿Qué número le toca a cada evento?

Tres máquinas y una cadena que da la vuelta: M0 avisa a M1, M1 a M2 y M2 de regreso a M0. Sobre cada flecha, el valor que viaja; junto a cada punto, el reloj del evento.



#	EVENTO	CÁLCULO	LC
1	M0 envía	`0 + 1`	**1**
2	M1 recibe	`max(0, 1) + 1`	**2**
3	M1 envía	`2 + 1`	**3**
4	M2 recibe	`max(0, 3) + 1`	**4**
5	M2 envía	`4 + 1`	**5**
6	M0 recibe	`max(1, 5) + 1`	**6**

En cada recepción gana el número que llegó.

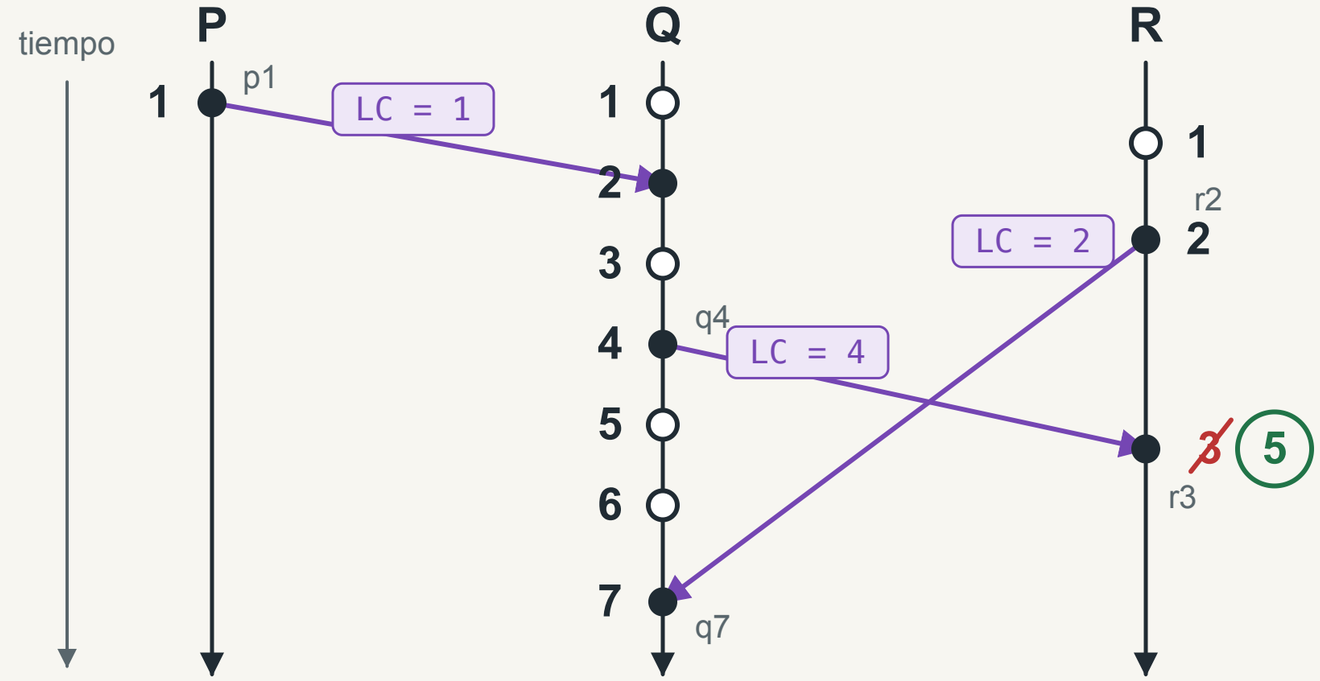
Una cadena de mensajes es una escalera: cada evento queda un escalón arriba.

La regla del máximo

¿Por qué r3 vale 5 y no 3?

Si R solo contara sus propios eventos, r3 valdría 3: menos que el 4 que le acaba de llegar.
El máximo obliga a quien recibe a quedar por encima de quien envió.

Basado en la figura 1 de Lamport (1978)



r3 · gana el mensaje

```
max(2, 4) + 1 = 5
```

✗ Solo contar: 2 + 1 = 3,
menor que el 4 que llegó
✓ Con el máximo: 5 > 4 y 5 > 2

q7 · gana el local

```
max(6, 2) + 1 = 7
```

El 2 llega tarde: Q ya iba en 6.
El mensaje no siempre manda.

↑ **Quien recibe nunca se queda por debajo de quien envió.**

Clock condition

¿Qué nos garantiza ese número?

Si A pasó antes que B, el reloj de A es estrictamente menor que el de B.

$$A \rightarrow B \Rightarrow LC(A) < LC(B)$$

A pudo influir en B — entonces — el reloj de A es menor

1 Mismo proceso

lo cubre el paso 2: + 1

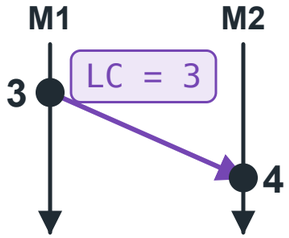


Cada evento suma 1.

✓ 2 < 3

2 Mensaje

lo cubre el paso 4: máximo + 1

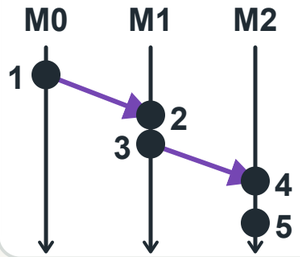


`max(0, 3) + 1 = 4`

✓ 3 < 4

3 Transitividad

se encadena sola



Si cada paso sube, la cadena entera sube.

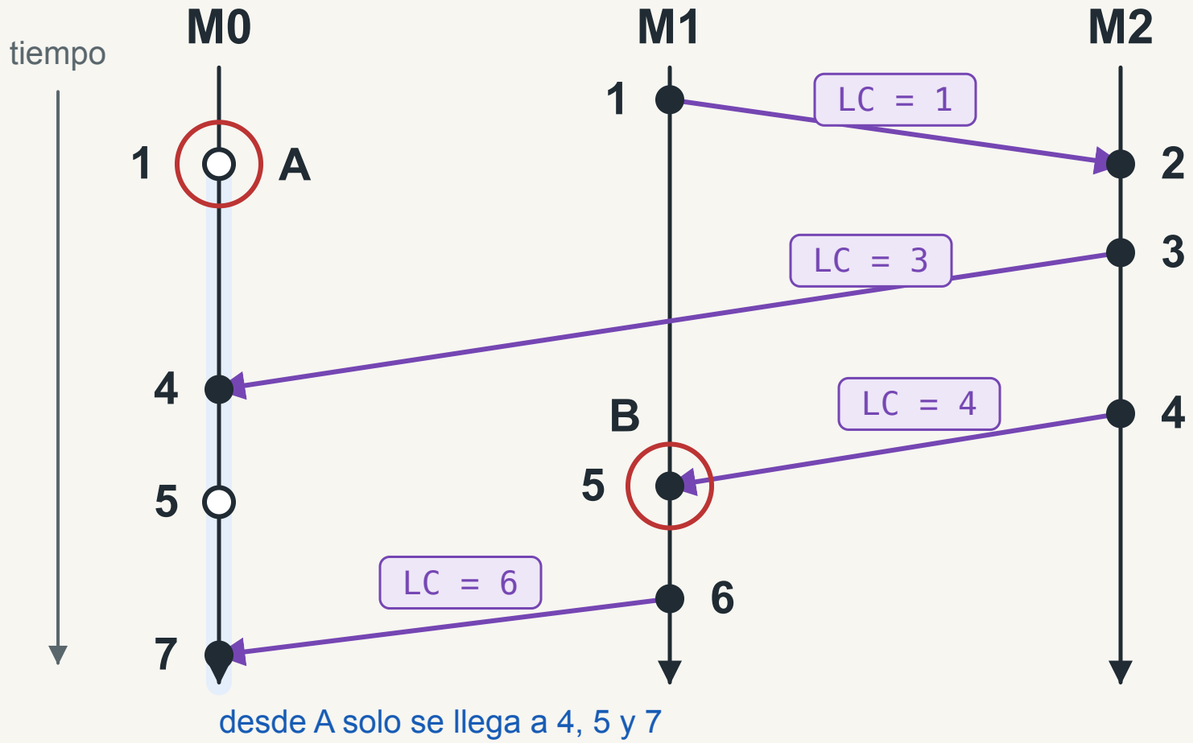
✓ 1 < 5

Si A pudo influir en B, el reloj de B siempre es mayor.

El contraejemplo

Si LC(A) < LC(B), ¿A pasó antes que B?

Al revés no vale: aquí LC(A) = 1 y LC(B) = 5, pero no hay camino de A a B.
A y B son concurrentes: A ‖ B.



desde A solo se llega a 4, 5 y 7

Los números

LC(A) = 1

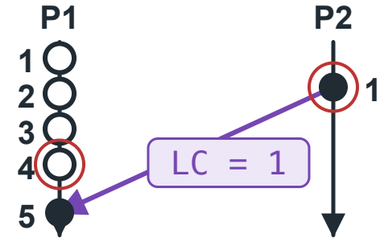
LC(B) = 5

✓ 1 < 5

A ‖ B

✗ ¿A → B? No: M0 nunca le envía nada a M1.

Versión mínima con dos procesos



1 < 4, y el envío de P2 no pasó antes que el 4.º evento de P1.

```
max(4, 1) + 1 = 5
```

⚠ **Un número menor no prueba que el evento pasó antes.**

Consistente, no caracteriza

¿En qué sentido vale la flecha?

Consistente: si hay causa, el número sube. Caracteriza: si el número sube, hay causa. Lamport cumple la primera, no la segunda.

CONSISTENTE CON LA CAUSALIDAD

$$A \rightarrow B \Rightarrow LC(A) < LC(B)$$

Lamport sí la cumple ✓

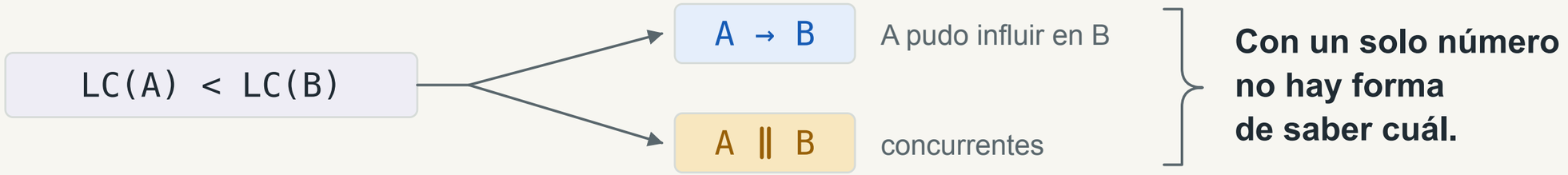
Ej.: el envío 3 y la recepción 4 de la cadena.

CARACTERIZA LA CAUSALIDAD

$$LC(A) < LC(B) \not\Rightarrow A \rightarrow B$$

Lamport no la cumple ✗

Ej.: el contraejemplo, $1 < 5$ y $A \parallel B$.



Lamport es consistente con la causalidad, pero no la caracteriza.

Descartar y ordenar

Si no caracterizan, ¿para qué sirven?

Dos usos que cuestan un solo número por mensaje:
descartar causas y fabricar un orden total que respeta la causalidad.

1 Descartar causas

$$\neg(LC(A) < LC(B)) \Rightarrow \neg(A \to B)$$

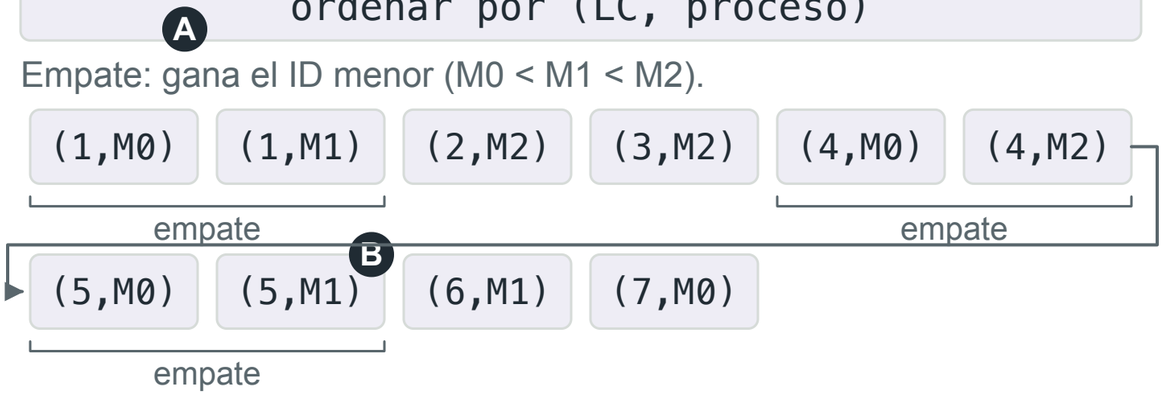
Si el reloj de A no es menor,
A no pudo causar B.

Quedan dos: $B \to A$ o $A \parallel B$

Del contraejemplo: LC(B) = 5
no es menor que LC(A) = 1
✓ B no pudo causar A

2 Orden total

`ordenar por (LC, proceso)`



✓ Si A → B, A sale antes en la fila.

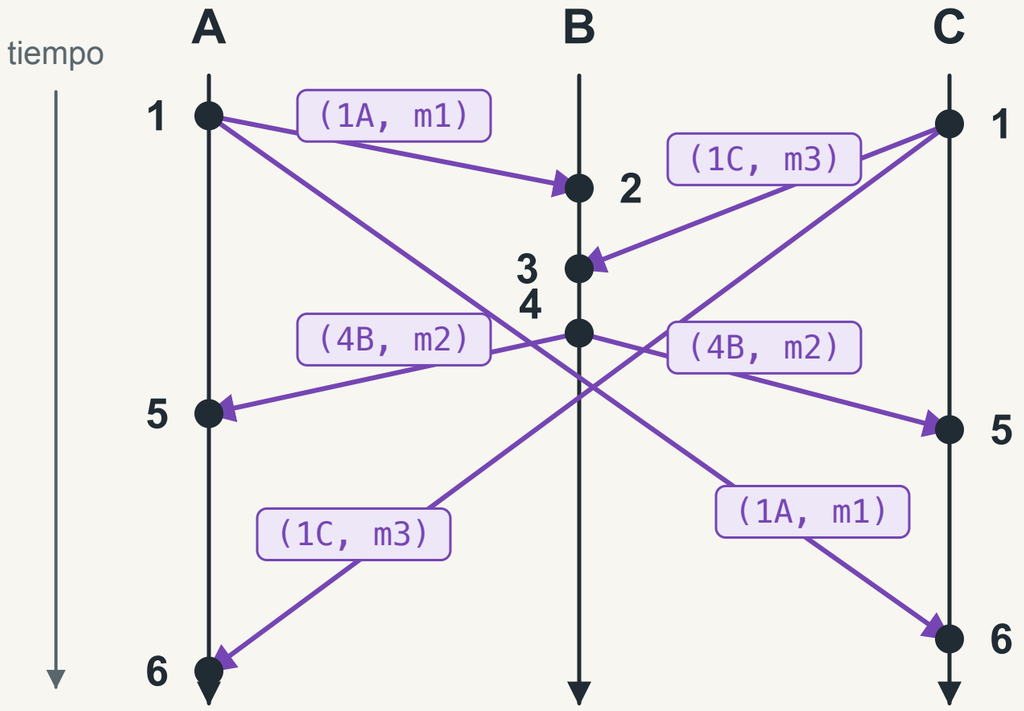
Los empatados eran concurrentes:
su orden es arbitrario, pero igual para todos.

Sirven para descartar causas y para que todos ordenen igual.

Chat sin servidor

¿Cómo ven todos la misma conversación sin nadie que ordene?

Cada mensaje viaja con su marca (reloj, proceso).
Cada quien muestra los mensajes ordenados por la marca, no por cómo llegaron.



A · m1
¿Quién viene a comer?

B · m2
Yo voy

C · m3
¡Qué calor!

responde a A

ASÍ LLEGAN A CADA UNO

A	(1A, m1)	(4B, m2)	(1C, m3)	✓ bien
B	(1A, m1)	(1C, m3)	(4B, m2)	✓ bien
C	(1C, m3)	(4B, m2)	(1A, m1)	✗ "Yo voy" antes de la pregunta

ORDENADO POR (RELOJ, PROCESO): IGUAL PARA TODOS

(1A, m1) (1C, m3) (4B, m2)

empate en 1: A antes que C

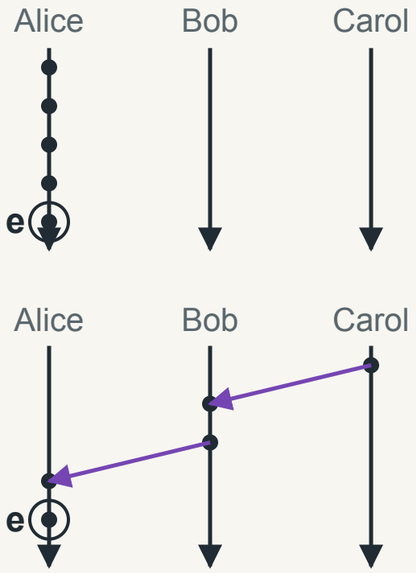
? Todos ordenan igual. ¿Y si quiero saber cuándo dos eventos son concurrentes?

Reloj vectorial

¿Qué sabe un 5 y qué sabe un [5,0,0]?

Un contador por proceso, juntos en un vector.
Cada posición dice cuántos eventos de ese proceso hay en el pasado del evento, contándolo a él.

Dos historias que terminan en el evento e



Historia 1
Alice, sola

Historia 2
Carol → Bob → Alice

Lamport: un número



5 eventos en la cadena

✗ mismo 5: no distingue



5 eventos en la cadena

Vector: uno por proceso

[5,0,0]
Alice Bob Carol

5 de Alice, 0 de Bob, 0 de Carol

✓ distintos: dice de quién

[2,2,1]
Alice Bob Carol

2 de Alice, 2 de Bob, 1 de Carol

Un número dice cuántos eventos; un vector dice cuántos de cada quién.

Algoritmo vectorial

¿Cómo se mantiene el vector al día?

Los mismos cuatro pasos de Lamport, con dos cambios: cada proceso suma 1 solo en su posición, y al recibir toma el máximo posición por posición.

1 Empieza en ceros

`[0,0,0]`

Alice Bob Carol

Todos acuerdan qué posición es de quién.

2 Todo evento: +1 en lo tuyo

Bob `[0,1,1]` —+1→ `[0,2,1]`

Envíos, recepciones y eventos internos.

3 Al enviar, el vector viaja

Bob —`[0,2,1]`→ Alice

Ya con el +1: enviar es un evento.

4 Al recibir: +1 y máximo

Alice recibe

`(1,0,0)` **max** `[0,2,1]` = `[1,2,1]`

+1 en tu posición y máximo con lo que llega.

Máximo punto a punto

	Alice	Bob	Carol
	[1	12	4]
	[7	0	2]
max	[**7**	**12**	**4**]

✓ cada posición, por separado

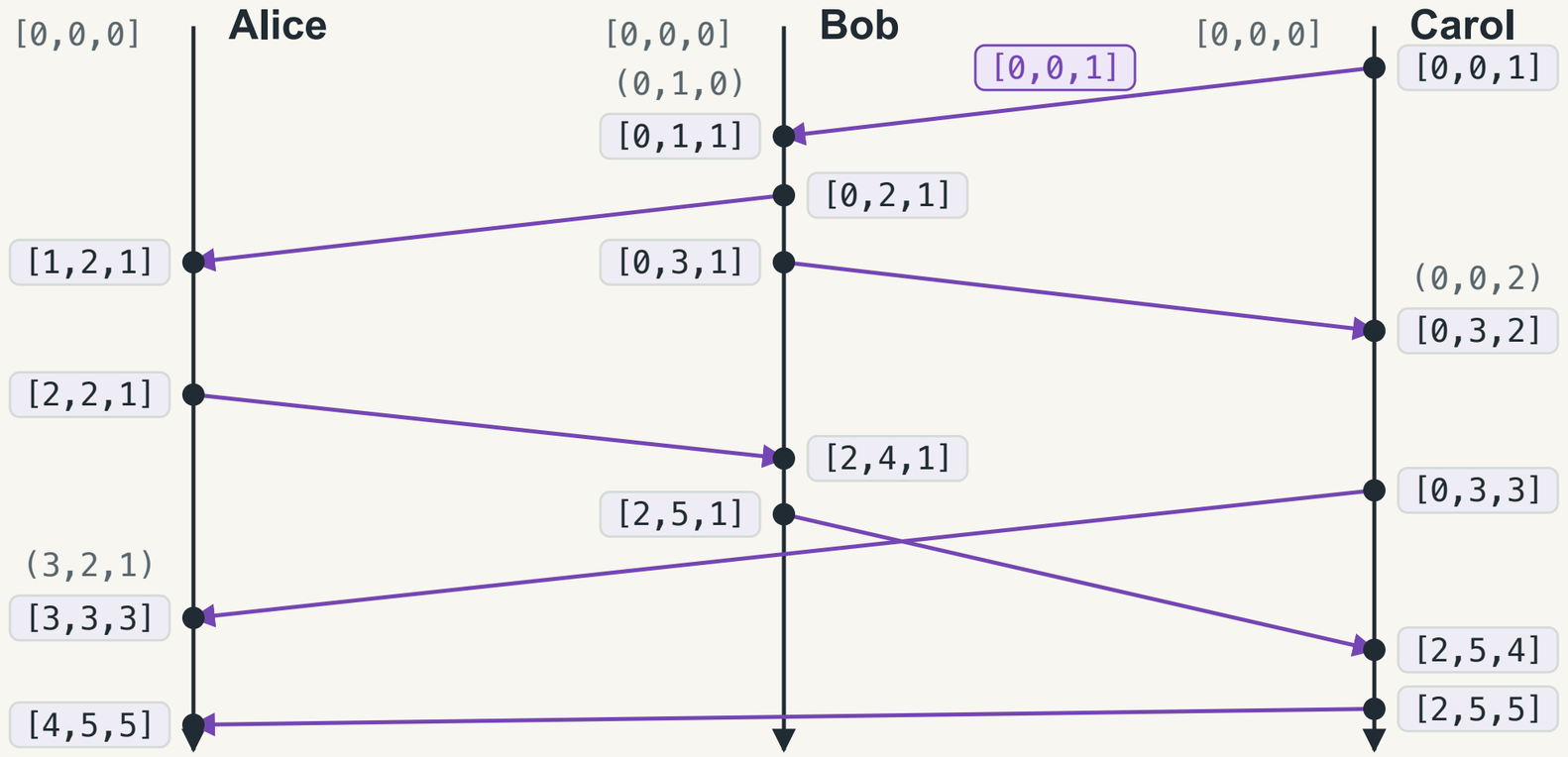
✗ no es elegir el vector "más grande"

Cada proceso cuenta lo suyo; al recibir, aprende lo que el otro sabía.

Corrida vectorial

Alice, Bob y Carol: ¿qué vector lleva cada evento?

Cada evento recibe su vector al ocurrir; el mensaje lleva el vector de su envío.



Cómo leerlo

Posiciones: [Alice, Bob, Carol]

Todos empiezan en [0,0,0]

→ lleva el vector de su envío

(0,1,0) = tras el +1, antes del max

Tres recepciones de cerca

Bob
`(0,1,0) max [0,0,1] = [0,1,1]`

Carol
`(0,0,2) max [0,3,1] = [0,3,2]`

Alice
`(3,2,1) max [0,3,3] = [3,3,3]`

El vector de un evento resume todo lo que ese evento pudo saber.

Comparar vectores

¿Cómo sé si un evento pasó antes que otro sin mirar el dibujo?

VC(X) < VC(Y): cada posición de X es ≤ que la de Y, y al menos una es distinta.
Si no vale ni VC(X) < VC(Y) ni VC(Y) < VC(X), X e Y son concurrentes.

A → B ⇄ VC(A) < VC(B)

Comparamos todo contra A = [2,4,1], el evento en que Bob recibe el mensaje de Alice

1 Menor: pasó antes

Alice envía a Bob

X	[2	2	1	]
	=	<	=	
A	[2	4	1	]

`[2,2,1] < [2,4,1]`

todas ≤ y una distinta: X → A

2 Mayor: pasó después

Carol recibe de Bob

X	[2	5	4	]
	=	>	>	
A	[2	4	1	]

`[2,5,4] > [2,4,1]`

todas ≥ y alguna distinta: A → X

3 Ni menor ni mayor

Alice recibe de Carol

X	[3	3	3	]
	>	<	>	
A	[2	4	1	]

`[3,3,3] ≮ [2,4,1] y ≯`

cada uno gana en algo: X ∥ A

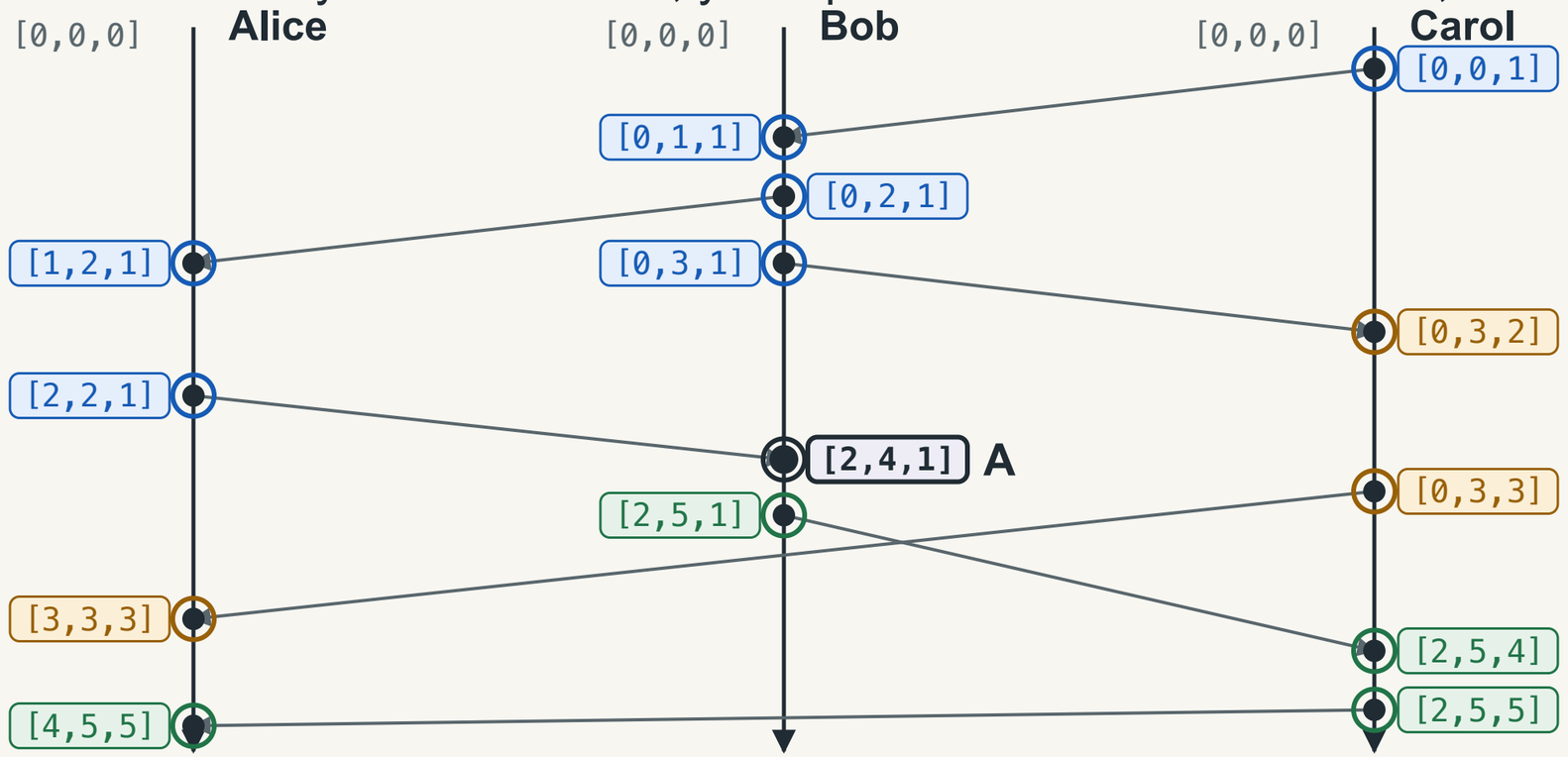
⚠ Ojo: < pide al menos una posición distinta. Dos eventos distintos nunca tienen el mismo vector.

Ya no hay que seguir flechas: basta comparar números, posición por posición.

Historia causal

¿Qué pudo influir en A = [2,4,1], y en qué pudo influir A?

Historia causal de A: los eventos que pudieron influir en A; son justo los de vector menor.
Los de vector mayor son su futuro, y los que no son ni lo uno ni lo otro, concurrentes.



Pasado de A · 6

[0,0,1] [0,1,1] [0,2,1]
[0,3,1] [1,2,1] [2,2,1]

cada posición ≤ [2,4,1]

Futuro de A · 4

[2,5,1] [2,5,4] [2,5,5]
[4,5,5]

cada posición ≥ [2,4,1]

Concurrentes con A · 3

[0,3,2] [0,3,3] [3,3,3]

mayor en una posición, menor en otra

Comparar contra [2,4,1] separa el pasado, el futuro y lo que no pudo influir.

Lamport contra vectores

¿Qué ganamos y qué pagamos por guardar N números?

Lamport es barato, pero solo sirve en un sentido.
El vector caracteriza la causalidad, a cambio de N números en cada mensaje.

	Lamport	`[5,0,0]` Vector
Qué guarda cada proceso	un contador	un vector de N contadores
Qué viaja en el mensaje	el contador, ya con el +1	el vector, ya con el +1
Qué garantiza	`A → B ⇒ LC(A) < LC(B)` solo en un sentido	`A → B ⇄ VC(A) < VC(B)`
¿Detecta concurrencia?	✗ **No:** Carol `5` y A `6` y son concurrentes	✓ **Sí:** `[0,3,2]` ∥ `[2,4,1]`
Costo por mensaje	**1 número**	**N números** ⚖ con procesos fijos y conocidos

Los vectores dicen qué pasó antes.
¿Y si un mensaje llega antes que su causa?

⚖ **Lamport descarta causas; el vector las caracteriza, a cambio de N números por mensaje.**

Entregar

¿Lo que llega se usa en cuanto llega?

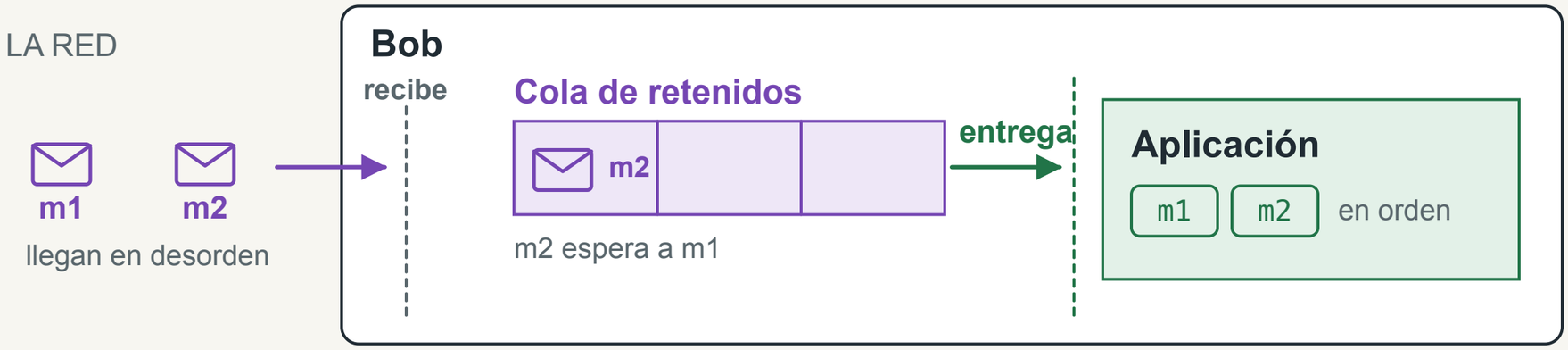
Entregar (deliver): pasarle a la aplicación un mensaje que ya llegó.
Entre recibir y entregar, el protocolo puede retenerlo en una cola.

1 Enviar `send`
Algo que haces.
Lo decide el emisor.

2 Recibir `receive`
Algo que te pasa.
Llega cuando la red quiera.

3 Entregar `deliver`
Algo que decides.
Lo decide el protocolo.

POR DENTRO DE BOB



TCP
recibe segmentos en desorden, entrega bytes en orden.

Consumidor de la S09
recibe el duplicado y decide no aplicarlo. ×2

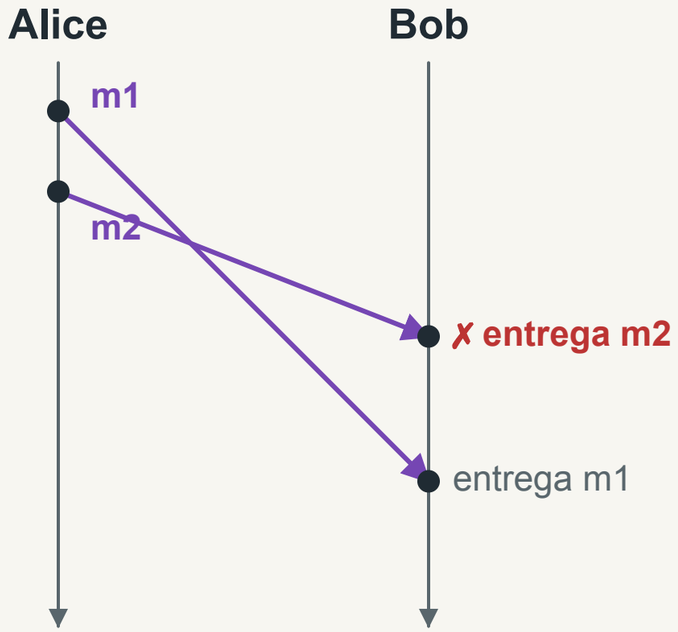
Recibir te pasa; entregar lo decides tú.

Entrega FIFO

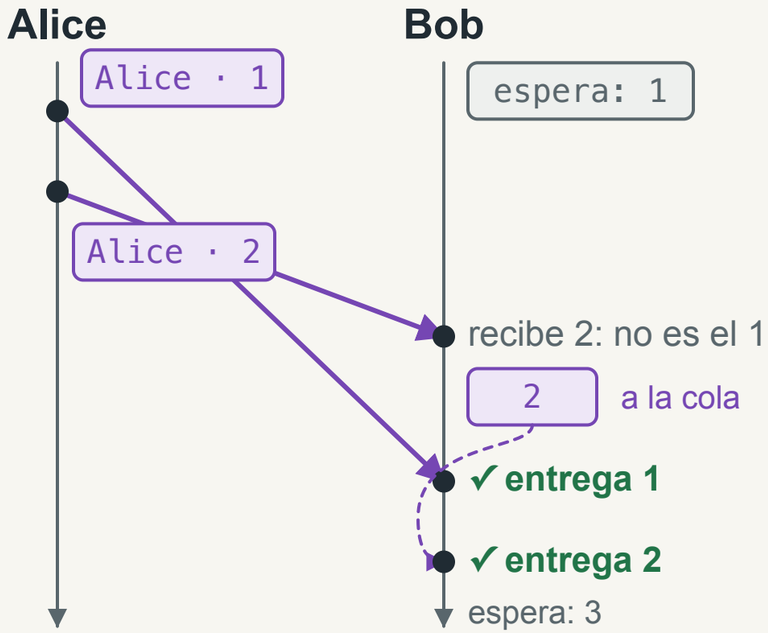
¿Y si m2 llega antes que m1?

Si un proceso envía m1 y después m2, quien entregue los dos entrega m1 primero.
Solo compara mensajes del mismo emisor al mismo receptor.

LA VIOLACIÓN



CON NÚMEROS DE SECUENCIA



LA REGLA

1. Cada mensaje lleva emisor y número.
2. El emisor suma 1 después de cada envío.
3. Bob entrega si es el anterior + 1; si no, lo encola.

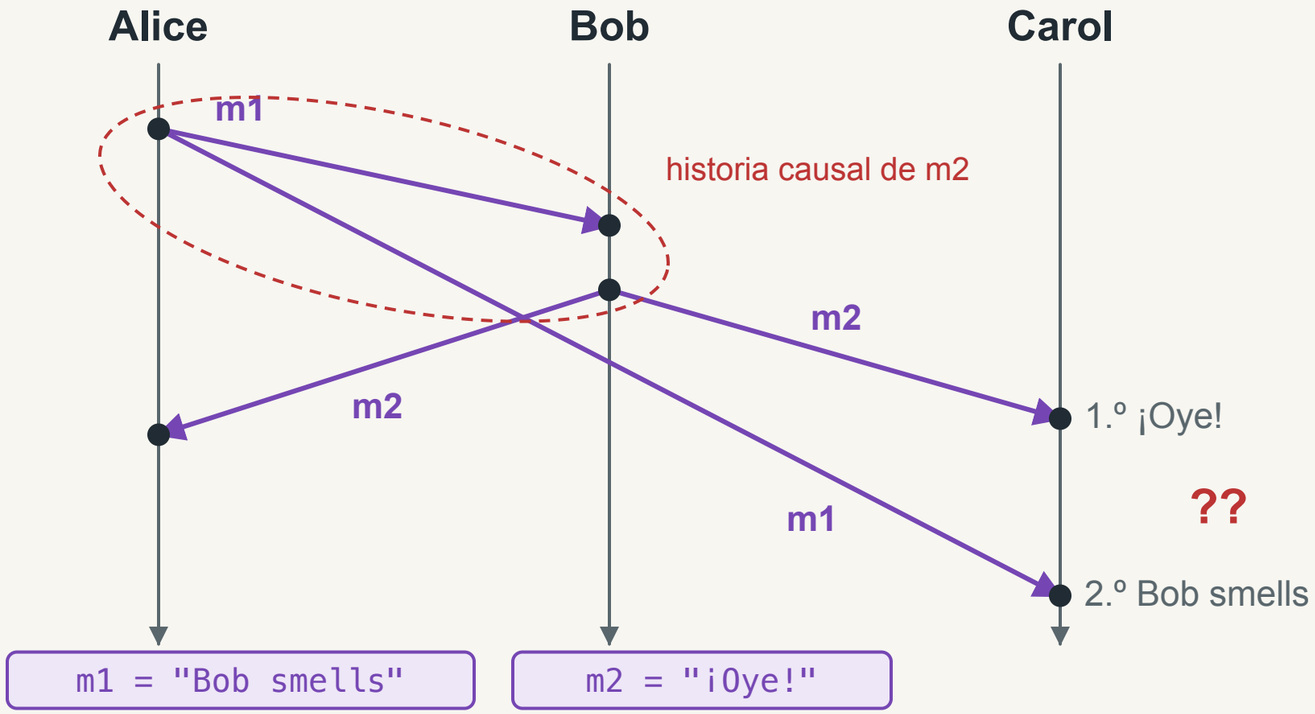
TCP ya te lo da
dentro de una conexión

FIFO cabe en un contador por emisor, y TCP ya lo trae.

Entrega causal

¿Y si la respuesta llega antes que el mensaje?

Si el envío de m1 pasó antes que el envío de m2, nadie entrega m2 antes que m1.
Incluye a FIFO y además ordena mensajes de emisores distintos.



¿Viola FIFO? No.
Nadie recibe dos mensajes del mismo emisor.

¿Viola causal? Sí.
Bob respondió después de recibir m1: m1 → m2.

¿TCP lo evita? No.
TCP ordena una conexión; aquí hay dos emisores.

Con vectores, Carol espera

`m2 · [1,1,0]` falta algo de Alice: la retiene

`m1 · [1,0,0]` llega: entrega m1 y luego m2 ✓

Vector = mensajes enviados por [Alice, Bob, Carol]

Para FIFO basta un contador; para causal hace falta un vector.

Relojes en producción

¿Dónde aparece lo que vimos hoy?

Casi nunca escribimos un reloj de Lamport: ya viene dentro de las herramientas.

Concepto	Tecnología	Para qué		Tipo
Relojes físicos y NTP	`Linux, chrony`	sincronizar la hora de los servidores		físico
Reloj monotónico	`Java, Go, Linux`	timeouts, latencias y duraciones		físico
Orden de mensajes	`Apache Kafka`	eventos en orden dentro de su partición	34	log
Tiempo lógico	`MongoDB`	sesiones con consistencia causal	36	lógico
Vectores de estado	`Yjs`	saber qué cambios le faltan a cada cliente		lógico
Reloj híbrido (HLC)	`CockroachDB`	marcas para transacciones distribuidas	37	híbrido
Incertidumbre acotada	`Google Spanner`	consistencia externa de transacciones	34	físico
Contexto de traza	`OpenTelemetry`	ver qué llamada provocó cuál	38	lógico

No todas dan la misma garantía de orden.

TrueTime contra Kafka

¿Compro un reloj mejor o dejo de preguntar la hora?

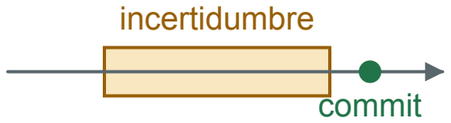
TrueTime acota la hora física con hardware y espera a que pase la duda.
Kafka no sincroniza relojes: numera cada mensaje dentro de su partición.

Google TrueTime
en la base de datos Spanner

Hardware
Relojes atómicos y GPS en cada centro de datos.



Commit wait
Espera a que pase la duda antes de confirmar.



Resultado
Lo que confirma después lleva una marca mayor.

`costo: hardware y espera`

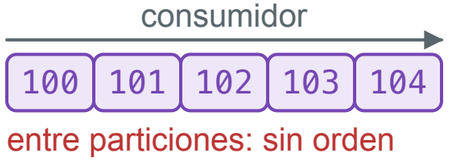
VS

Apache Kafka
el log de la S09

Software
No sincroniza relojes: numera cada mensaje.



Orden por partición
Quien la lea, la lee en el mismo orden.



Resultado
Corre en hardware común.

`costo: llaves y particiones`

TrueTime compra un reloj mejor; Kafka deja de preguntar la hora.

Lamport en Java

¿Cuánto código es un reloj de Lamport?

Un contador atómico: sube en cada evento y en cada envío.
Al recibir, toma el máximo entre el suyo y el del mensaje, y suma uno.

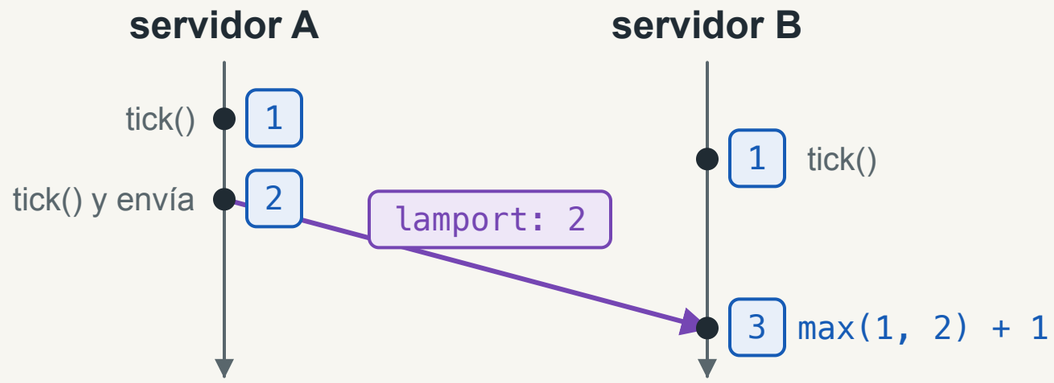
```java
public class LamportClock {
    private final AtomicLong reloj = new AtomicLong(0);

    // evento local o envío
    public long tick() {
        return reloj.incrementAndGet();
    }

    // recepción de un mensaje con su marca
    public long recibir(long remota) {
        return reloj.updateAndGet(
            local -> Math.max(local, remota) + 1);
    }
}
```

no detecta concurrentes

reinicio: persistir contador



viaja dentro del evento (S09)

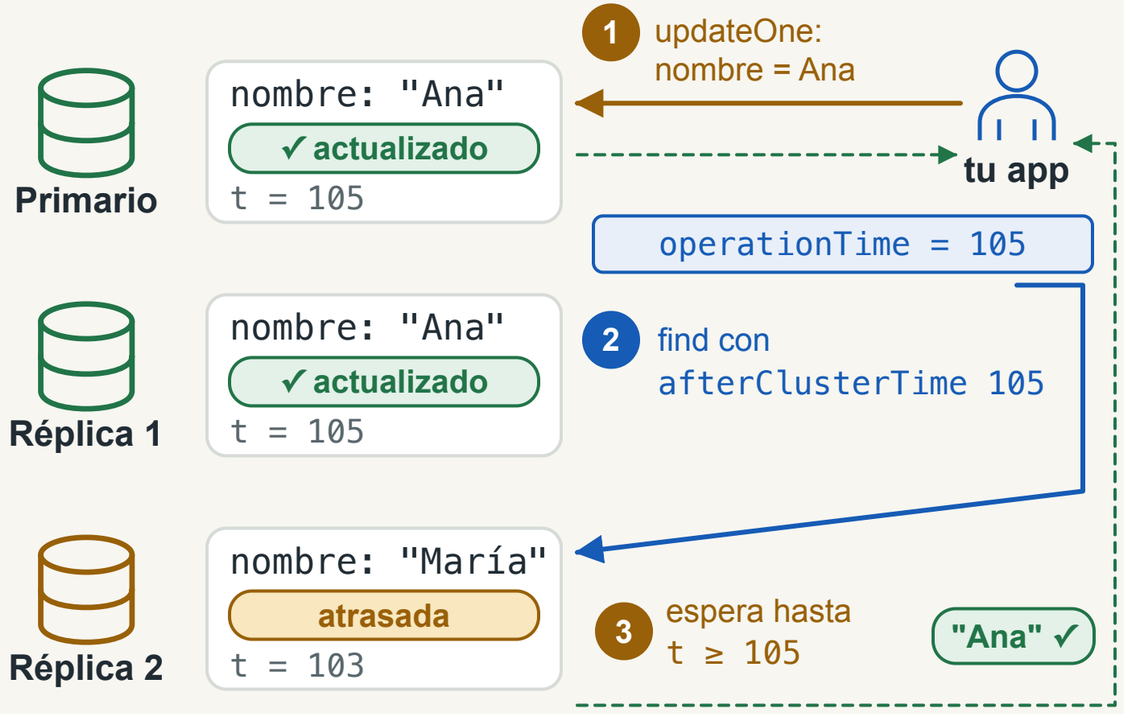
```
{ "transferenciaId": "TRF-123",
  "evento": "TransferenciaRealizada",
  "lamport": 2,
  "origen": "transferencia" }
```

⚠ **Ordena causas; no detecta concurrentes.**

Sesión causal

¿Por qué no veo el nombre que acabo de cambiar?

MongoDB lleva en la sesión la marca lógica de tu última operación.
La réplica que atiende tu lectura espera a haber visto esa marca.




```
ClientSession s = mongoClient.startSession(
    ClientSessionOptions.builder()
        .causallyConsistent(true).build());

var usuarios = db.getCollection("usuarios")
    .withReadConcern(ReadConcern.MAJORITY)
    .withWriteConcern(WriteConcern.MAJORITY)
    .withReadPreference(ReadPreference.secondary());

usuarios.updateOne(s, eq("_id", "u123"),
    set("nombre", "Ana"));
usuarios.find(s, eq("_id", "u123")).first(); // "Ana"
```

lees lo que escribiste | **lecturas que no retroceden** | **escrituras en orden**

Read-your-writes sin reloj de pared: la sesión carga su marca lógica.

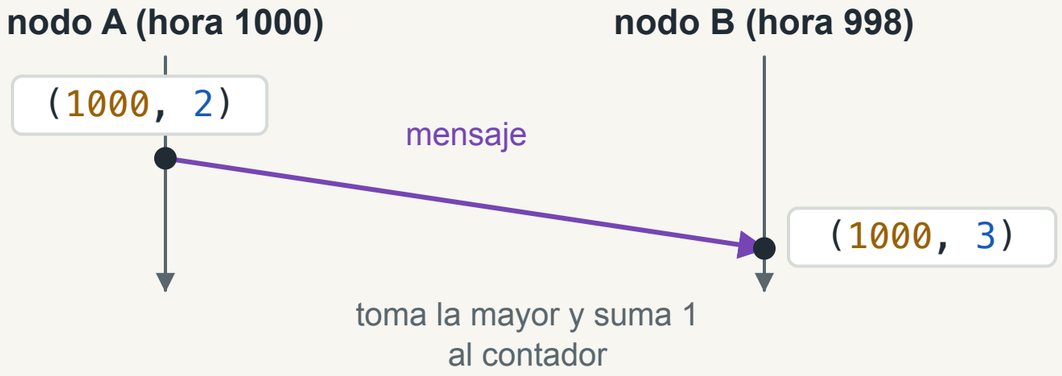
Reloj híbrido

¿Y si juntamos la hora con un contador?

HLC = (tiempo físico, contador lógico): cerca de la hora real y sin romper la causalidad.

`HLC = (tiempo físico, contador)`

Evento	Hora del nodo	HLC	Por qué
A	1000	(1000, 0)	hora nueva: contador en 0
B	1000	(1000, 1)	misma hora: sube el contador
C	1000	(1000, 2)	misma hora: sube otra vez
D	1001	(1001, 0)	avanzó la hora: contador en 0



No es un vector:
dos marcas HLC distintas no prueban que una causó a la otra.

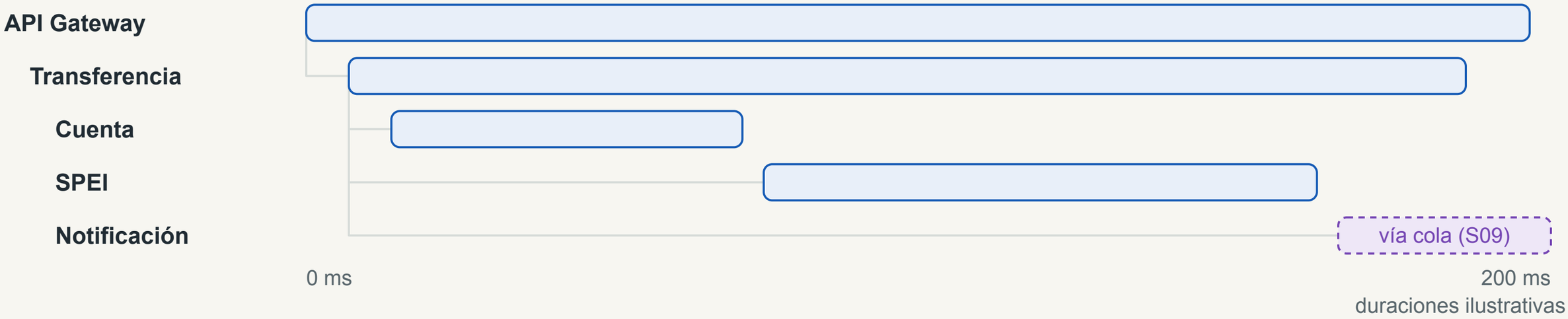
Lo usan:
CockroachDB, MongoDB y YugabyteDB, junto con replicación y control de concurrencia.

Hora legible para humanos, orden seguro para los mensajes.

Traza distribuida

¿Qué llamada provocó cuál?

OpenTelemetry propaga un contexto en cada llamada: el mismo traceId para toda la petición y un spanId por operación, con su padre. Así se reconstruyen las causas.



encabezado HTTP (W3C Trace Context)

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

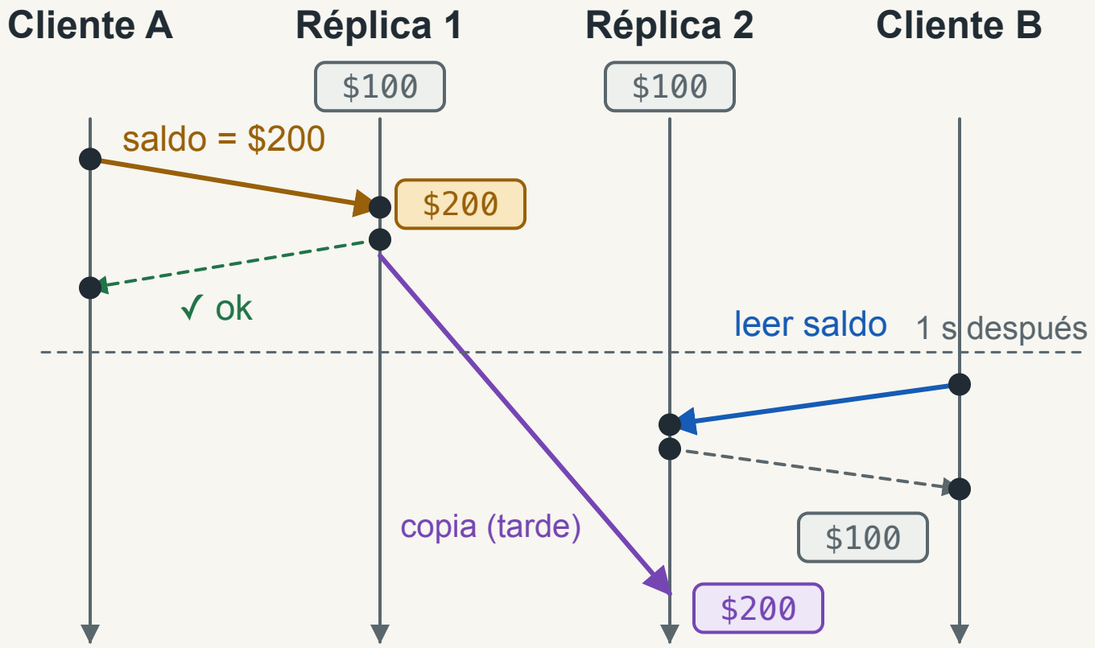
versión · traceId: toda la petición · spanId del padre · muestreo

Causalidad para observar, no para ordenar escrituras.

Causal contra linealizable

¿Ordenar por causas basta para leer el saldo nuevo?

La causalidad es una relación entre eventos: ¿A pudo influir en B?
La linealizabilidad es una garantía: el sistema actúa como una sola copia, en tiempo real.



	Causal	Linealizable
Se apoya en	happens-before (→)	el tiempo real
Promete	ningún efecto sin su causa	una sola copia, siempre al día
B lee 1 s después	puede ver $100 ✓	**debe ver $200;** $100 sería ✗
Se logra con	relojes lógicos, sin coordinar	líder o consenso (S06)
Costo	bajo: cada réplica sigue sola	alto: esperar a la mayoría

⚖ **Los relojes lógicos dan orden causal; ser una sola copia exige coordinarse.**

Antes, después, concurrentes

¿Cómo sabemos qué pasó primero sin un reloj compartido?

Tres relojes, tres preguntas: cada uno sabe un poco más que el anterior.

1 Relojes físicos

¿Qué hora es?

10:03 — 10:00

desfase

NTP acerca los relojes,
pero nunca los iguala.

2 Lamport

42

¿Pudo A causar B?

`LC(A) = 1` < `LC(B) = 5`

¿A → B? no se sabe

Si A → B, entonces LC(A) < LC(B).
Al revés no se cumple.

3 Vectoriales

¿Antes, después o concurrentes?

`[2,4,1]` || `[3,3,3]`

concurrentes

Comparan entrada por entrada.
Cuestan un número por proceso.

No siempre necesitamos saber qué hora es; necesitamos saber qué pasó antes, qué pasó después y qué eventos son concurrentes.

El orden se construye; no se lee del reloj.

Su propia historia

¿Para qué estudiar relojes si las bases de datos ya los traen?

1 Ya viene construido



Muchas herramientas ya implementan estos relojes por dentro.

2 No son lo mismo



Orden de mensajes, causalidad y linealizabilidad son garantías distintas.

orden · causal · linealizable

3 Saber qué te dan



Entender qué garantiza tu tecnología y qué le toca a tu aplicación.



Un sistema distribuido no tiene un único reloj perfecto, pero puede tener mecanismos que le permiten razonar correctamente sobre su propia historia.